

0.1 Thiết kế kiến trúc

0.1.1 Lựa chọn kiến trúc phần mềm

Hệ thống được xây dựng theo kiến trúc phân tầng kết hợp mô hình “Simulation–Visualization” (mô phỏng–hiển thị), trong đó phần mô phỏng giao thông chạy trên SUMO (điều khiển bằng TraCI trong Python) và phần hiển thị/điều khiển chạy trên Unity (C#). Hai phần giao tiếp với nhau thông qua kết nối TCP socket theo cơ chế truyền thông điệp dạng JSON (có ký hiệu kết thúc gói tin <END> để tách thông điệp trên TCP stream).

Lý do lựa chọn kiến trúc này:

- **Tách biệt trách nhiệm:** SUMO xử lý mô phỏng vi mô (phương tiện, đèn giao thông, người đi bộ), Unity tập trung vào trực quan 3D, giao diện người dùng và tương tác.
- **Dễ mở rộng:** có thể bổ sung thêm loại đối tượng (xe buýt, xe ưu tiên), thêm chỉ số thống kê (thời gian hành trình, mật độ) mà không ảnh hưởng mạnh đến phần còn lại.
- **Tương thích công cụ:** SUMO đã có TraCI hỗ trợ điều khiển/đọc trạng thái theo bước mô phỏng; Unity thuận lợi cho hiển thị thời gian thực.

Trong cách ánh xạ theo hướng MVC ở mức khái niệm:

- **Model:** trạng thái mô phỏng trong SUMO (vehicle, traffic light, pedestrian) và các tệp mạng (.net.xml, .rou.xml, .sumocfg).
- **View:** các prefab/manager trong Unity để dựng hình đường, xe, đèn và hiển thị UI.
- **Controller:** lớp giao tiếp TCP và bộ xử lý lệnh (Pause/Resume/End) để đồng bộ mô phỏng.

0.1.2 Thiết kế tổng quan

Kiến trúc tổng quan gồm 3 tầng chính:

1. **Tầng mô phỏng (Python + SUMO/TraCI):** khởi tạo mạng giao thông, chạy mô phỏng theo step, trích xuất trạng thái và ghi thống kê kết quả.
2. **Tầng tích hợp (TCP + JSON protocol):** đóng gói dữ liệu trạng thái (đèn giao thông, xe, người đi bộ), xử lý phân mảnh/ghép gói trên TCP stream bằng marker <END>.
3. **Tầng hiển thị (Unity):** nhận dữ liệu theo thời gian thực, cập nhật/khởi tạo đối tượng trong scene; cung cấp UI cho người dùng điều khiển và kết thúc mô

phòng.

Hình minh họa kiến trúc tổng quan (tự vẽ theo mô tả):

Unity (C#) → [JSON/TCP]port 5050 Python Main → TraCI SUMO.

Unity gửi lệnh điều khiển (Pause/Resume/End) qua port điều khiển, Python phản hồi bằng dữ liệu trạng thái theo bước mô phỏng.

Hình 0.1: Kiến trúc tổng quan của hệ thống mô phỏng–hiển thị

0.1.3 Thiết kế chi tiết gói

Các gói/module chính và nhiệm vụ:

- **Python/HelloWorld:**

- `main.py`: điểm vào chương trình; khởi tạo server socket, nhận lệnh từ Unity, chạy vòng lặp TraCI và gửi dữ liệu trạng thái mỗi step.
- `network.py`: đóng gói giao thức truyền thông; đảm bảo tách thông điệp theo <END> để khắc phục đặc tính stream của TCP.
- `result.py`: ghi log hành trình xe ra CSV và tổng hợp thống kê ra JSON.
- `Traffic/*`: các module đọc trạng thái (xe, đèn, người đi bộ, đường) từ TraCI và chuẩn hóa về dạng JSON.
- `SUMO_xml/*`: sinh tệp cấu hình/mạng SUMO (node/edge/route/net/sumocfg), trong đó có chức năng sinh mạng từ mô cung.

- **Unity/Assets/Scripts:**

- `TrafficDataListener`: kết nối tới Python, gửi thông điệp `SimulationReady` và lắng nghe trạng thái mô phỏng theo thời gian thực.
- `RoadDataListener`: gửi `RoadDataRequest` để tải dữ liệu đường/nút giao một lần khi khởi động scene.
- `*Manager` (`VehicleManager`, `TrafficLightManager`, `PedestrianManager`): cập nhật/khởi tạo đối tượng dựa trên danh sách dữ liệu nhận được.
- `UI/OptionUI`: UI tùy chọn và gửi lệnh kết thúc mô phỏng (`SimulationEnd`).

0.2 Thiết kế chi tiết

0.2.1 Thiết kế giao diện

Ứng dụng hướng tới hiển thị mô phỏng thời gian thực trong Unity. Giao diện được thiết kế tối giản, tập trung vào:

- **Màn hình hiển thị chính:** camera quan sát mạng đường, xe và đèn giao thông;

các đối tượng được cập nhật theo dữ liệu từ SUMO.

- **Bảng tùy chọn:** bật/tắt UI trong khi chạy, cho phép kết thúc mô phỏng an toàn (gửi lệnh về phía Python).

Giả định môi trường chạy phổ biến là PC, độ phân giải khuyến nghị 1920×1080 (Full HD) hoặc cao hơn; UI được thiết kế theo nguyên tắc dễ đọc, hạn chế chữ nhỏ và đảm bảo thao tác bằng chuột/bàn phím.

0.2.2 Thiết kế lớp

Trong phạm vi báo cáo, trình bày chi tiết một số lớp/module chủ đạo.

Nhóm lớp Python (mô phỏng và ghi log)

VehicleTripCsvLogger (trong `result.py`):

- Trách nhiệm: ghi nhận thời điểm xe xuất phát/đến đích theo step mô phỏng; ghi CSV và xuất thống kê trung bình ra JSON.
- Thuộc tính chính: `_depart_step_by_vehicle`, `_travel_steps_sum`, `_travel_steps_count`.
- Phương thức chính: `open()`, `on_step(step, departed, arrived)`, `close()`.

network.py (giao thức TCP):

- Trách nhiệm: tách/gộp thông điệp trên TCP bằng marker `<END>`; tránh lỗi khi một lần `recv` chứa nhiều thông điệp hoặc chỉ chứa một phần thông điệp.
- Hàm chính: `send_data()`, `receive_data()`, `receive_message()`, `server_thread()`.

Nhóm lớp Unity (nhận dữ liệu và cập nhật scene)

TrafficDataListener:

- Trách nhiệm: kết nối TCP đến Python, gửi `SimulationReady`, đọc JSON trạng thái; đẩy xử lý sang main thread để cập nhật scene an toàn.
- Phương thức chính: `StartListening()`, `ListenForData()`, `HandleData(json)`.

RoadDataListener:

- Trách nhiệm: gửi yêu cầu `RoadDataRequest` và nhận dữ liệu đường/nút giao 1 lần; lưu vào `RoadDataSO` để các hệ thống khác sử dụng.

TrafficLightManager:

- Trách nhiệm: quản lý tập đối tượng đèn giao thông trong scene; nếu đã tồn tại

thì cập nhật trạng thái/vị trí, nếu chưa thì tạo mới từ prefab.

Luồng thông điệp (mô tả dạng trình tự)

Use case 1 — Unity yêu cầu dữ liệu đường (RoadData):

1. Unity khởi chạy `RoadDataListener` và mở kết nối TCP tới Python.
2. Unity gửi chuỗi `RoadDataRequest`.
3. Python trả về JSON mô tả edge/junction/crossing (đóng gói và kết thúc bằng `<END>`).
4. Unity parse JSON và lưu vào `RoadDataSO`; sau đó dừng listener.

Use case 2 — Đồng bộ trạng thái mô phỏng theo thời gian thực:

1. Unity gửi `SimulationReady` để bắt đầu vòng lặp mô phỏng.
2. Python chạy TraCI theo step; mỗi step đọc danh sách xe/đèn/người đi bộ và gửi về Unity.
3. Unity nhận JSON, phân rã thành danh sách và gọi các manager để cập nhật đối tượng trong scene.
4. Khi người dùng thoát ứng dụng, Unity gửi `Simulation end` để yêu cầu dừng mô phỏng.

0.2.3 Thiết kế cơ sở dữ liệu

Hệ thống không sử dụng cơ sở dữ liệu theo nghĩa truyền thống. Dữ liệu được lưu dưới dạng tệp:

- **Dữ liệu cấu hình/mạng mô phỏng:** các tệp XML của SUMO (`.nod.xml`, `.edg.xml`, `.net.xml`, `.rou.xml`, `.sumocfg`).
- **Dữ liệu kết quả thực nghiệm:** CSV ghi thời gian hành trình và JSON tổng hợp thống kê theo lần chạy (trong thư mục `result/`).

Thiết kế này phù hợp vì mục tiêu là mô phỏng/hiển thị và thu thập thống kê theo từng lần chạy, không cần truy vấn phức tạp hay đa người dùng.

0.3 Xây dựng ứng dụng

0.3.1 Thư viện và công cụ sử dụng

Bảng 1 liệt kê các công cụ/chính sách phiên bản sử dụng trong dự án.

0.3.2 Kết quả đạt được

Các sản phẩm/dữ liệu đầu ra chính:

- **Bộ sinh bản đồ/mạng SUMO:** tạo node–edge từ bản đồ thành phố hoặc từ

Mục đích	Công cụ	Phiên bản/Ghi chú
Mô phỏng giao thông vi mô	Eclipse SUMO	1.24.0 (tạo file .sumocfg thể hiện version)
Điều khiển SUMO	TraCI (Python API)	đi kèm SUMO 1.24.0
Ngôn ngữ mô phỏng/tích hợp	Python	(điền theo máy: chạy python -version)
Trực quan 3D + UI	Unity Editor	6000.3.0f1
Parse JSON (Unity)	Newtonsoft.Json	com.unity.nuget.newtonsoft-json 3.2.2
Môi trường phát triển	Visual Studio Code	(điền theo máy: About VS Code)
Quản lý phiên bản	Git	(điền theo máy: git -version)
Hệ điều hành chạy thử nghiệm	Windows	(10/11 tùy máy triển khai)

Bảng 1: Danh sách thư viện và công cụ sử dụng

mê cung; sinh ra các tệp .nod.xml, .edg.xml, .net.xml, .rou.xml, .sumocfg.

- **Ứng dụng Unity hiển thị mô phỏng:** nhận dữ liệu thời gian thực từ Python và cập nhật đối tượng (xe, đèn, người đi bộ).
- **Bộ ghi log kết quả:** xuất CSV hành trình xe theo step và JSON tổng hợp theo từng lần chạy.

Thống kê nhanh mã nguồn (tính trên các phần do nhóm tự viết, loại trừ thư viện/.venv):

- Python: 20 file, 3045 dòng, dung lượng khoảng 109.5 KB.
- Unity C# (Assets/Scripts): 72 file, 4261 dòng, dung lượng khoảng 126.9 KB.
- Tệp cấu hình/mạng SUMO (HelloWorld/SUMO_xml): 6 tệp, tổng dung lượng khoảng 10.34 MB.
- Bản build Windows minh họa: UnityBuild/TestGR1.1.exe khoảng 0.64 MB (chưa tính toàn bộ thư mục build).

0.3.3 Minh họa các chức năng chính

Các chức năng chính đã thực hiện:

1. **Sinh mạng đường từ dữ liệu đầu vào:** hỗ trợ sinh mạng từ tệp bản đồ thành phố và sinh mạng từ mê cung (tạo node/edge theo cấu trúc lưới).

2. **Chạy mô phỏng SUMO bằng TraCI:** điều khiển mô phỏng theo step và đọc trạng thái xe/đèn/người đi bộ.
3. **Đồng bộ dữ liệu sang Unity theo thời gian thực:** gửi gói JSON theo TCP; Unity nhận và cập nhật scene.
4. **Điều khiển mô phỏng:** hỗ trợ tạm dừng/tiếp tục và kết thúc mô phỏng từ Unity.
5. **Xuất kết quả thực nghiệm:** ghi log hành trình xe và thống kê trung bình theo lần chạy.

Chèn ảnh minh họa 1: Scene Unity đang hiển thị xe/đèn giao thông theo dữ liệu SUMO.

Hình 0.2: Minh họa trực quan mô phỏng trên Unity

Chèn ảnh minh họa 2: SUMO-GUI đang chạy mô phỏng và cấu hình delay (ví dụ 110ms) để đồng bộ quan sát.

Hình 0.3: Minh họa mô phỏng trên SUMO-GUI

0.4 Kiểm thử

Do dự án có tính tích hợp cao (SUMO & Unity), kiểm thử được thực hiện theo hai hướng: (i) kiểm thử chức năng thử công theo use case và (ii) kiểm thử hộp đen cho các module xử lý dữ liệu/giao thức.

Các trường hợp kiểm thử tiêu biểu

TC	Mục tiêu	Dữ liệu vào / Kỳ vọng
TC01	Tách thông điệp TCP đúng theo <END>	Gửi liên tiếp 2 JSON trong 1 lần sendall; kỳ vọng phía nhận tách đúng 2 thông điệp, không lẫn chuỗi.
TC02	Nhận RoadData một lần	Unity gửi RoadDataRequest; kỳ vọng nhận JSON hợp lệ và lưu vào RoadDataSO, sau đó listener dừng.
TC03	Cập nhật trạng thái đèn giao thông	SUMO trả về chuỗi trạng thái r/y/g/G; kỳ vọng Unity ánh xạ về trạng thái hiển thị tương ứng (đỏ/vàng/xanh).

Bảng 2: Một số trường hợp kiểm thử chính

Kết quả: các ca kiểm thử trên đạt yêu cầu trong điều kiện chạy thử nghiệm. Trường hợp sai lệch đồng bộ (giật/nhảy vị trí) chủ yếu xuất hiện khi delay quan sát

trong SUMO-GUI đặt quá thấp; khi tăng delay (ví dụ khoảng 110ms) thì quan sát ổn định hơn.

0.5 Triển khai

Môi trường triển khai

Hệ thống được triển khai thử nghiệm trên máy cá nhân Windows với 2 tiến trình chính:

- **Python + SUMO**: chạy script chính để khởi tạo server và điều khiển TraCI.
- **Unity (Editor hoặc bản build)**: chạy scene nhận dữ liệu và hiển thị.

Quy trình chạy thử nghiệm

Quy trình triển khai thử nghiệm (tóm tắt theo README):

1. Thêm dự án Unity vào Unity Hub và mở project.
2. Chạy script Python `main.py` để chuẩn bị kết nối và khởi chạy mô phỏng.
3. Khi SUMO-GUI mở, đặt delay quan sát phù hợp (ví dụ 110) và chạy mô phỏng.
4. Chạy Unity (Play mode hoặc bản build) để bắt đầu nhận dữ liệu và hiển thị.

Các cổng TCP sử dụng theo cấu hình hiện tại:

- Port 5050: Unity kết nối để nhận dữ liệu RoadData/TrafficData.
- Port điều khiển (ví dụ 5054): Unity gửi lệnh kết thúc mô phỏng (`Simulation end`).

Trong phạm vi thử nghiệm, hệ thống chạy ổn định theo mô hình “1 máy – 1 người dùng”; dữ liệu kết quả được lưu cục bộ thành CSV/JSON để phục vụ phân tích.